

# Veda ERP

## Complete API Reference

---

*79 Route Files | 13 Logical Sections*

Base URL: /api | Auth: JWT in httpOnly cookie

Roles: admin, operator, accountant

# Overview

This document provides a comprehensive reference for all API endpoints in the Veda Enterprises ERP system. The API follows a consistent RESTful pattern across all modules. Every collection module exposes a standard set of CRUD endpoints: a base route for listing (GET) and creating (POST), an [id] route for fetching (GET), updating (PUT), and deleting (DELETE) individual records, and a bulk-delete sub-route (POST) for batch operations. All routes use force-dynamic rendering with Cache-Control: no-store headers, ensuring that every request reflects the latest database state without any caching layer interference.

Authentication is handled via JWT tokens stored in httpOnly cookies. The system supports three roles with graduated permissions: admin (full access to all operations including destructive ones), operator (can create and edit records but is restricted from certain financial and administrative operations), and accountant (read-mostly role that is explicitly blocked from most destructive operations like bulk-delete and data clearing). Several modules implement cross-module auto-sync logic, where creating, updating, or deleting a record in one module automatically triggers side effects in related modules to maintain data consistency.

The API is built on Next.js App Router with MongoDB as the data store. Path parameters using [id] accept MongoDB ObjectId strings. Date normalization handles multiple formats including DD-MM-YYYY, DD/MM/YY, Excel serial numbers, and ISO strings. The system also includes an AI integration layer supporting OpenAI and Groq providers for intelligent data parsing, a full database backup and restore system with merge semantics, and a comprehensive Excel import pipeline supporting 12 modules.

## Table of Contents

**Section 1: Authentication & Users**

**Section 2: Company & Settings**

**Section 3: Dashboard & Reports**

**Section 4: Customers**

**Section 5: Production & Stock**

**Section 6: Orders & Dispatch**

**Section 7: Daily Sell & Payments**

**Section 8: Purchases**

**Section 9: Misc Expenses**

**Section 10: Bills / Invoicing**

**Section 11: Excel Import**

**Section 12: Admin & Debug**

**Section 13: AI**

**Summary Tables**

# Section 1: Authentication & Users

## POST /api/auth/login

**Auth:** Open

**Purpose:** User login with email+password; issues JWT in httpOnly cookie (maxAge 86400s/24h).

**Body:** { email: string, password: string } (max 256 chars each)

**Response:** { message: 'Login successful', user: { id, name, email, role } } + sets token cookie. 401 on bad creds. Uses dummy bcrypt compare to prevent user enumeration via timing.

## POST /api/auth/init

**Auth:** Open

**Purpose:** First-run system initialization. Seeds the initial admin and Veda Enterprises company record. Refuses if any users exist. Optional FIRST\_RUN\_KEY env gate via X-First-Run-Key header or ?key= query.

**Response:** { message, user: { id, name, email, role } } (201) or { message: 'Users already exist...' } (400).

## GET /api/auth/me

**Auth:** requireSession

**Purpose:** Returns the currently logged-in user session payload.

**Response:** { user: { userId, email, role, name } } or 401.

## POST /api/auth/reset-admin

**Auth:** EMERGENCY\_RESET\_KEY

**Purpose:** Emergency admin password reset. Resets admin user to admin123, reactivates account, or creates fresh admin if none exists. Gated by EMERGENCY\_RESET\_KEY env var via header or query.

**Response:** { message, credentials: { email, password: 'admin123' }, userId, action: 'reset'|'created', note? }.

## POST /api/auth/forgot-password/request-otp

**Auth:** Open

**Purpose:** Generates 6-digit OTP, bcrypt-hashes it, stores in PasswordReset collection with 10-min TTL, emails it. Always returns 200 (no user enumeration). Rate-limited: 1 OTP/email/60s.

**Body:** { email: string }

**Response:** { message, email, expiryMinutes: 10, emailConfigured?, devPreview? } (200) or 429 with cooldownSeconds.

## POST /api/auth/forgot-password/verify-otp

**Auth:** Open

**Purpose:** Validates OTP against latest unused PasswordReset doc. Max 5 attempts. On success issues 10-min signed resetToken JWT.

**Body:** { email: string, otp: string } (6 digits)

**Response:** { message, resetToken, email } or 401/410/429 with attemptsLeft.

## POST /api/auth/forgot-password/reset

**Auth:** Open (resetToken)

**Purpose:** Sets new password. Verifies resetToken JWT, checks PasswordReset doc is still verified && !used, updates password (bcrypt rounds=12), marks doc as used.

**Body:** { email, resetToken, newPassword, confirmPassword } (min 6, max 256 chars)

**Response:** { message, email } or 401/403/404.

## GET /api/users

**Auth:** requireAdmin

**Purpose:** List all users (passwords stripped), newest first.

**Response:** { users: User[] }.

### POST /api/users

**Auth:** requireAdmin

**Purpose:** Create a new user.

**Body:** { name, email (valid), password (>=6 chars), role: 'admin'|'operator'|'accountant', active?: boolean }

**Response:** { user: UserWithoutPassword } (201). 400 on duplicate email/invalid role/short password.

### GET /api/users/[id]

**Auth:** requireAdmin

**Purpose:** Fetch one user by ObjectId.

**Response:** { user: UserWithoutPassword } or 404.

### PUT /api/users/[id]

**Auth:** requireAdmin

**Purpose:** Update user fields. Optional password change (re-hashed at rounds=12).

**Body:** partial { name?, email?, role?, active?, password? }

**Response:** { user: UserWithoutPassword }.

### DELETE /api/users/[id]

**Auth:** requireAdmin

**Purpose:** Delete a user. Blocks self-delete and deleting the last active admin.

**Response:** { message: 'User deleted successfully' } or 400 (self-delete / last admin).

### POST /api/users/bulk-delete

**Auth:** Admin only

**Purpose:** Bulk delete users. Blocks self-delete and last-admin deletion.

**Body:** { ids: string[] } (non-empty)

**Response:** { message, deletedCount, requestedCount } or 404.

### POST /api/users/bulk-update

**Auth:** Admin only

**Purpose:** Bulk activate/deactivate users. Blocks self-deactivation and last-admin deactivation.

**Body:** { ids: string[], active: boolean }

**Response:** { message, modifiedCount, matchedCount, requestedCount, active }.

## Section 2: Company & Settings

### GET /api/company

**Auth:** requireSession

**Purpose:** Fetch the single Company record (creates one with Veda defaults if missing; backfills empty contact fields; migrates stale tagline strings).

**Response:** { company: Company } (fields: name, tagline, address, city, state, pincode, phone, email, gstNumber, panNumber, logoUrl, primaryColor, bankName, bankAccount, bankIfsc, invoicePrefix, dispatchPrefix, orderPrefix, terms, signatureName, setupComplete).

### PUT /api/company

**Auth:** requireAdmin

**Purpose:** Update company fields. Auto-marks setupComplete=true once name+address+phone+gstNumber are all present.

**Body:** partial of any Company field

**Response:** { company: Company }.

## Section 3: Dashboard & Reports

### GET /api/dashboard

**Auth:** requireSession

**Purpose:** Classic dashboard aggregate. Today production, total stock, today dispatch, pending orders, outstanding payments, monthly sales/profit/expense, recent productions + dispatches, monthly production + expense chart data.

**Response:** { todayProduction, totalStock, todayDispatch, pendingOrders, outstandingPayments, monthlySales, monthlyProfit, recentProductions, recentDispatches, monthlyProductionData, monthlyExpenseData }.

### GET /api/dashboard/stats

**Auth:** requireSession

**Purpose:** Single-call KPI endpoint (replaces 11 separate dashboard calls). Aggregates today production/sales/labour/customer-payments/tractor-remaining/dust/cement/hardner/electricity/factory-stuff + stock totals + net cash flow.

**Response:** { todayProduction, todaySales, todayLabourPayments, todayCustomerPayments, totalTractorRemaining, todayDustPurchase, todayCementPurchase, todayHardner, todayElectricity, todayFactoryStuff, totalStock, totalStockCement, totalExpensesToday, netCashFlow }.

### GET /api/reports

**Auth:** requireSession

**Purpose:** Multi-mode report generator. Switch on ?type= (sales|production|stock|profit-loss|outstanding|customer-ledger). Optional month/from/to date filters.

**Body:** ?type=, ?month=YYYY-MM, ?from=YYYY-MM-DD, ?to=YYYY-MM-DD

**Response:** Response varies by type: sales={ data,totalSales }, production={ data,totalProduced,byBrickType }, stock={ data,totalCurrentStock,totalOpeningStock,lowStockItems }, profit-loss={ reportType,month,totalRevenue,totalExpenses,netProfit,expensesByCategory }, outstanding={ data,totalOutstanding }, customer-ledger={ customerLedger }.

### GET /api (root)

**Auth:** Open

**Purpose:** Health check endpoint.

**Response:** { status: 'ok', message: 'Veda Enterprises ERP API is running' }.

## Section 4: Customers

### GET /api/customers

**Auth:** requireSession

**Purpose:** Paginated customer list with search.

**Body:** ?search= (regex on name/mobile), ?page= (default 1), ?limit= (default 100, max 500)

**Response:** { customers: Customer[], total, page, limit, totalPages }.

### POST /api/customers

**Auth:** requireRole(['admin','operator'])  
**Purpose:** Create a new customer.  
**Body:** { name, mobile, gstNumber?, address?, creditLimit? }  
**Response:** { customer: Customer } (201).

### GET /api/customers/[id]

**Auth:** requireSession  
**Purpose:** Fetch one customer by ObjectId.  
**Response:** { customer: Customer } or 404.

### PUT /api/customers/[id]

**Auth:** requireRole(['admin','operator'])  
**Purpose:** Update customer fields.  
**Body:** partial { name?, mobile?, gstNumber?, address?, creditLimit? }  
**Response:** { customer: Customer }.

### DELETE /api/customers/[id]

**Auth:** requireRole(['admin','operator'])  
**Purpose:** Delete a customer.  
**Response:** { message: 'Customer deleted successfully' }.

### POST /api/customers/bulk-delete

**Auth:** Admin/Operator  
**Purpose:** Bulk delete customers; nulls out customerId references in Order/Payment/CustomerPayment/Production/Dispatch.  
**Body:** { ids: string[] }  
**Response:** { message, deletedCount, requestedCount }.

### GET /api/customers/[id]/history

**Auth:** requireSession  
**Purpose:** Customer ledger: orders, dispatches, payments, customer payments + merged sorted timeline[] and summary aggregations.  
**Response:** { customer, summary, orders, dispatches, payments, customerPayments, timeline }.

### GET /api/customers/[id]/bill-history

**Auth:** requireSession  
**Purpose:** Bill generation context: customer record, productions, dispatches, bills, orders, payments, aggregated product totals.  
**Response:** { customer, productions, dispatches, bills, orders, payments, productFields, summary: {...} }.

## Section 5: Production & Stock

### GET /api/production

**Auth:** requireSession  
**Purpose:** List all production entries (date-desc).  
**Body:** ?date=YYYY-MM-DD  
**Response:** { productions: Production[] }.

### POST /api/production

**Auth:** requireRole(['admin','operator'])

**Purpose:** Create production entry. Calls syncStockForDate(date) to update Stock snapshot.

**Body:** { date, customerId?, cement?, zigZagGrey80?, zigZagRed80?, zigZagYellow80?, zigZagGrey60?, zigZagRed60?, zigZagYellow60?, curveStone?, chequreTile?, dumbleGrey80?, dumbleRed80?, dumbleYellow80?, transportationCharge?, remarks? }

**Response:** { production: Production } (201).

## DELETE /api/production?all=true

**Auth:** requireAdmin

**Purpose:** Wipes ALL production + ALL stock entries. ?all=true REQUIRED.

**Body:** ?all=true

**Response:** { message, deletedCount }.

## GET /api/production/[id]

**Auth:** requireSession

**Purpose:** Fetch one production entry.

**Response:** { production: Production } or 404.

## PUT /api/production/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Update production entry. Re-syncs Stock snapshot for the date.

**Body:** partial Production fields

**Response:** { production: Production }.

## DELETE /api/production/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Delete production entry. Re-syncs Stock snapshot for the date.

**Response:** { message: 'Production entry deleted successfully' }.

## POST /api/production/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete production. Re-aggregates Stock snapshots for every touched date.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount, stockResyncedDates }.

## GET /api/stock

**Auth:** requireSession

**Purpose:** List all stock entries (date-desc).

**Response:** { stocks: Stock[] }.

## POST /api/stock

**Auth:** requireAdmin

**Purpose:** Create stock entry OR bulk-delete (dispatches by body shape).

**Body:** Create: { date, cement?, ... } / Bulk-delete: { ids: string[] }

**Response:** { stock: Stock } (201) or { message, deletedCount }.

## DELETE /api/stock?all=true

**Auth:** requireAdmin

**Purpose:** Wipe every stock entry. ?all=true REQUIRED.

**Body:** ?all=true

**Response:** { message, deletedCount }.

### GET /api/stock/[id]

**Auth:** requireSession

**Purpose:** Fetch one stock entry.

**Response:** { stock: Stock } or 404.

### PUT /api/stock/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Update stock entry. Accepts backward-compat aliases like zigZagGrey80mm.

**Body:** partial Stock fields

**Response:** { stock: Stock }.

### DELETE /api/stock/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Delete stock entry.

**Response:** { message: 'Stock entry deleted successfully' }.

### GET /api/stock/summary

**Auth:** requireSession

**Purpose:** Item-wise stock summary: one row per product (12 products) showing totalProduction, sellItem, availableQuantity, previousYearStock, latestDate, latestQuantity, productionDays. Single-pass aggregation.

**Response:** { summary: SummaryRow[] }.

## Section 6: Orders & Dispatch

### GET /api/orders

**Auth:** requireSession

**Purpose:** List all orders with populated customer field.

**Response:** { orders: Order[] }.

### POST /api/orders

**Auth:** requireRole(['admin','operator'])

**Purpose:** Create order. Generates orderNumber ORD-####. Supports single brickType or multi-item orders.

**Body:** { customerId, deliveryDate, status?, brickType?, quantity?, rate?, amount?, items?: [{ description, hsn?, unit?, quantity, rate, amount? }] }

**Response:** { order: Order } (201).

### GET /api/orders/[id]

**Auth:** requireSession

**Purpose:** Fetch one order.

**Response:** { order: Order }.

### PUT /api/orders/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Update order. If items[] provided, recomputes summary fields.

**Body:** partial { customerId?, brickType?, quantity?, rate?, amount?, deliveryDate?, status?, items? }

**Response:** { order: Order }.



### **DELETE** /api/orders/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Delete order. Does NOT cascade-delete linked Bills/Dispatches.

**Response:** { message: 'Order deleted successfully' }.

### **POST** /api/orders/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete orders.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

### **GET** /api/dispatch

**Auth:** requireSession

**Purpose:** List all dispatches with customer + order populated.

**Response:** { dispatches: Dispatch[] }.

### **POST** /api/dispatch

**Auth:** requireRole(['admin','operator'])

**Purpose:** Create dispatch. Auto-decrements Stock.currentStock for the matching brickType.

**Body:** { customerId, orderId?, truckNumber, driverName?, quantity, brickType, date }

**Response:** { dispatch: Dispatch } (201).

### **GET** /api/dispatch/[id]

**Auth:** requireSession

**Purpose:** Fetch one dispatch.

**Response:** { dispatch: Dispatch }.

### **PUT** /api/dispatch/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Update dispatch.

**Body:** partial dispatch fields

**Response:** { dispatch: Dispatch }.

### **DELETE** /api/dispatch/[id]

**Auth:** requireRole(['admin','operator'])

**Purpose:** Delete dispatch. Restores Stock.currentStock by the deleted quantity.

**Response:** { message: 'Dispatch deleted successfully' }.

### **POST** /api/dispatch/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete dispatches.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

## Section 7: Daily Sell & Payments

## GET /api/daily-sell

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** List all daily sell entries (date-desc).

**Response:** { dailySells: DailySell[] }.

## POST /api/daily-sell

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create daily sell entry OR bulk-delete (admin-only branch). Create triggers syncAllFromDailySell() which mirrors into Customer, Order, CustomerPayment, Payment, TractorPayment + Stock.

**Body:** Create: { date, customerName, address?, contactNumber?, product?, quantity?, rate?, amount, transporterName?, transporterFair?, receivedAmount?, remarks? } / Bulk-delete: { ids: string[] } (admin only)

**Response:** { dailySell: DailySell } (201) or { message, deletedCount }.

## DELETE /api/daily-sell?all=true

**Auth:** requireAdmin

**Purpose:** Wipe all daily sell entries. Runs cleanupDailySellLinks() for every record. ?all=true REQUIRED.

**Body:** ?all=true

**Response:** { message, deletedCount }.

## GET /api/daily-sell/[id]

**Auth:** requireSession

**Purpose:** Fetch one daily sell entry.

**Response:** { dailySell: DailySell } or 404.

## PUT /api/daily-sell/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update daily sell entry. Auto-recomputes pendingAmount. Re-runs cleanup+sync so mirrors stay fresh.

**Body:** partial { date, customerName, address, contactNumber, product, quantity, rate, amount, transporterName, transporterFair, receivedAmount, remarks }

**Response:** { dailySell: DailySell }.

## DELETE /api/daily-sell/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete daily sell entry. Runs cleanupDailySellLinks() to remove mirrored records.

**Response:** { message: 'Daily sell entry deleted successfully' }.

## GET /api/customer-payment

**Auth:** requireSession

**Purpose:** List all customer payments (date-desc).

**Response:** { customerPayments: CustomerPayment[] }.

## POST /api/customer-payment

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create customer payment.

**Body:** { date, name, address?, amount, remarks? }

**Response:** { customerPayment: CustomerPayment } (201).

## GET /api/customer-payment/[id]

**Auth:** requireSession

**Purpose:** Fetch one customer payment.

**Response:** { customerPayment: CustomerPayment }.

### **PUT /api/customer-payment/[id]**

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update customer payment.

**Body:** partial { date?, name?, address?, amount?, remarks? }

**Response:** { customerPayment: CustomerPayment }.

### **DELETE /api/customer-payment/[id]**

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete customer payment.

**Response:** { message: 'customerPayment entry deleted successfully' }.

### **POST /api/customer-payment/bulk-delete**

**Auth:** Admin/Operator

**Purpose:** Bulk delete customer payments.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

### **GET /api/labour-payment**

**Auth:** requireSession

**Purpose:** List all labour payments (date-desc).

**Response:** { labourPayments: LabourPayment[] }.

### **POST /api/labour-payment**

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create labour payment.

**Body:** { date, name, address?, amount, remarks? }

**Response:** { labourPayment: LabourPayment } (201).

### **GET /api/labour-payment/[id]**

**Auth:** requireSession

**Purpose:** Fetch one.

**Response:** { labourPayment: LabourPayment }.

### **PUT /api/labour-payment/[id]**

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update labour payment.

**Body:** partial { date?, name?, address?, amount?, remarks? }

**Response:** { labourPayment: LabourPayment }.

### **DELETE /api/labour-payment/[id]**

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete labour payment.

**Response:** { message: 'labourPayment entry deleted successfully' }.

### **POST /api/labour-payment/bulk-delete**

**Auth:** Admin/Operator

**Purpose:** Bulk delete labour payments.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

## GET /api/tractor-payment

**Auth:** requireSession

**Purpose:** List all tractor payments (date-desc).

**Response:** { tractorPayments: TractorPayment[] }.

## POST /api/tractor-payment

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create tractor payment. Computes  $\text{totalAmount} = \text{quantityTon} \times \text{rate}$  and  $\text{remainingAmount} = \text{totalAmount} - \text{paidAmount}$ .

**Body:** { date, vendorName, quantityTon, rate, paidAmount?, remarks?, type?: "tractor"|"transporter", linkedDailySellId? }

**Response:** { tractorPayment: TractorPayment } (201).

## GET /api/tractor-payment/[id]

**Auth:** requireSession

**Purpose:** Fetch one.

**Response:** { tractorPayment: TractorPayment }.

## PUT /api/tractor-payment/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update tractor payment.

**Body:** partial tractor payment fields

**Response:** { tractorPayment: TractorPayment }.

## DELETE /api/tractor-payment/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete tractor payment.

**Response:** { message: 'tractorPayment entry deleted successfully' }.

## POST /api/tractor-payment/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete tractor payments.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

## GET /api/payments

**Auth:** requireSession

**Purpose:** List all payments with customer populated. Links to Bill via billId/billNumber and mirrors to CustomerPayment via customerPaymentId.

**Response:** { payments: Payment[] }.

## POST /api/payments

**Auth:** requireRole(['admin','accountant'])

**Purpose:** Create payment. If billId provided, recomputes Bills paidAmount/balanceAmount/status. Mirrors into CustomerPayment.

**Body:** { customerId, paymentType, amount, date, remarks?, billId? }

**Response:** { payment: Payment } (201).

## GET /api/payments/[id]

**Auth:** requireSession

**Purpose:** Fetch one payment.

**Response:** { payment: Payment }.

### PUT /api/payments/[id]

**Auth:** requireRole(['admin','accountant'])

**Purpose:** Update payment. If billId changed, re-syncs both old and new Bills. Mirrors update into CustomerPayment.

**Body:** partial { customerId?, paymentType?, amount?, date?, remarks?, billId? }

**Response:** { payment: Payment }.

### DELETE /api/payments/[id]

**Auth:** requireRole(['admin','accountant'])

**Purpose:** Delete payment. Runs syncDeleteCustomerPayment(id) to remove mirror. Re-syncs linked Bills paidAmount.

**Response:** { message: 'Payment deleted successfully' }.

### POST /api/payments/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete payments. Runs syncDeleteCustomerPayment(id) for each.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

## Section 8: Purchases

### GET /api/dust-purchase

**Auth:** requireSession

**Purpose:** List dust purchases (date-desc).

**Response:** { dustPurchases: DustPurchase[] }.

### POST /api/dust-purchase

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create dust purchase. Computes totalAmount = quantity x rate.

**Body:** { date, vendorName, cementName?, quantity, rate, paidAmount?, transportationCharge?, gst?, remarks? }

**Response:** { dustPurchase: DustPurchase } (201).

### GET /api/dust-purchase/[id]

**Auth:** requireSession

**Purpose:** Fetch one.

**Response:** { dustPurchase: DustPurchase }.

### PUT /api/dust-purchase/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update dust purchase.

**Body:** partial fields

**Response:** { dustPurchase: DustPurchase }.

### DELETE /api/dust-purchase/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete dust purchase.  
**Response:** { message: 'dustPurchase entry deleted successfully' }.

### POST /api/dust-purchase/bulk-delete

**Auth:** Admin/Operator  
**Purpose:** Bulk delete dust purchases.  
**Body:** { ids: string[] }  
**Response:** { message, deletedCount, requestedCount }.

### GET /api/cement-purchase

**Auth:** requireSession  
**Purpose:** List cement purchases (date-desc).  
**Response:** { cementPurchases: CementPurchase[] }.

### POST /api/cement-purchase

**Auth:** requireRole(['admin','operator','accountant'])  
**Purpose:** Create cement purchase. Computes totalAmount = quantity x rate.  
**Body:** { date, vendorName, itemName?, quantity, rate, paidAmount?, transportationCharge?, gst?, remarks? }  
**Response:** { cementPurchase: CementPurchase } (201).

### GET /api/cement-purchase/[id]

**Auth:** requireSession  
**Purpose:** Fetch one.  
**Response:** { cementPurchase: CementPurchase }.

### PUT /api/cement-purchase/[id]

**Auth:** requireRole(['admin','operator','accountant'])  
**Purpose:** Update cement purchase.  
**Body:** partial fields  
**Response:** { cementPurchase: CementPurchase }.

### DELETE /api/cement-purchase/[id]

**Auth:** requireRole(['admin','operator','accountant'])  
**Purpose:** Delete cement purchase.  
**Response:** { message: 'cementPurchase entry deleted successfully' }.

### POST /api/cement-purchase/bulk-delete

**Auth:** Admin/Operator  
**Purpose:** Bulk delete cement purchases.  
**Body:** { ids: string[] }  
**Response:** { message, deletedCount, requestedCount }.

## Section 9: Misc Expenses

### GET /api/hardner

**Auth:** requireSession  
**Purpose:** List hardners (date-desc).

**Response:** { hardners: Hardner[] }.

### POST /api/hardner

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create hardner entry (only 2 fields!).

**Body:** { date, amount }

**Response:** { hardner: Hardner } (201).

### GET /api/hardner/[id]

**Auth:** requireSession

**Purpose:** Fetch one.

**Response:** { hardner: Hardner }.

### PUT /api/hardner/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update hardner.

**Body:** partial { date?, amount? }

**Response:** { hardner: Hardner }.

### DELETE /api/hardner/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete hardner.

**Response:** { message: 'hardner entry deleted successfully' }.

### POST /api/hardner/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete hardners.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

### GET /api/electricity

**Auth:** requireSession

**Purpose:** List electricity entries (date-desc). Note: response key is electricitys (not electricities).

**Response:** { electricitys: Electricity[] }.

### POST /api/electricity

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create electricity entry.

**Body:** { date, name?, work?, amount, remarks? }

**Response:** { electricity: Electricity } (201).

### GET /api/electricity/[id]

**Auth:** requireSession

**Purpose:** Fetch one.

**Response:** { electricity: Electricity }.

### PUT /api/electricity/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update electricity.

**Body:** partial { date?, name?, work?, amount?, remarks? }

**Response:** { electricity: Electricity }.

### DELETE /api/electricity/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete electricity.

**Response:** { message: 'electricity entry deleted successfully' }.

### POST /api/electricity/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete electricity entries.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

### GET /api/factory-stuff

**Auth:** requireSession

**Purpose:** List factory stuff entries (date-desc).

**Response:** { factoryStuffs: FactoryStuff[] }.

### POST /api/factory-stuff

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Create factory stuff entry.

**Body:** { date, itemName, quantity?, amount, remarks? }

**Response:** { factoryStuff: FactoryStuff } (201).

### GET /api/factory-stuff/[id]

**Auth:** requireSession

**Purpose:** Fetch one.

**Response:** { factoryStuff: FactoryStuff }.

### PUT /api/factory-stuff/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Update factory stuff.

**Body:** partial { date?, itemName?, quantity?, amount?, remarks? }

**Response:** { factoryStuff: FactoryStuff }.

### DELETE /api/factory-stuff/[id]

**Auth:** requireRole(['admin','operator','accountant'])

**Purpose:** Delete factory stuff.

**Response:** { message: 'factoryStuff entry deleted successfully' }.

### POST /api/factory-stuff/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete factory stuff entries.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

### GET /api/expenses

**Auth:** requireSession



**Purpose:** List expenses (date-desc). Supports category and date filters.

**Body:** ?category= (exact), ?date= (single OR from,to comma-pair)

**Response:** { expenses: Expense[] }.

## POST /api/expenses

**Auth:** requireRole(['admin','accountant'])

**Purpose:** Create expense. Note: operators NOT allowed (admin+accountant only).

**Body:** { category, amount, date, description? }

**Response:** { expense: Expense } (201).

## GET /api/expenses/[id]

**Auth:** requireSession

**Purpose:** Fetch one.

**Response:** { expense: Expense }.

## PUT /api/expenses/[id]

**Auth:** requireRole(['admin','accountant'])

**Purpose:** Update expense.

**Body:** partial { category?, amount?, date?, description? }

**Response:** { expense: Expense }.

## DELETE /api/expenses/[id]

**Auth:** requireRole(['admin','accountant'])

**Purpose:** Delete expense.

**Response:** { message: 'Expense deleted successfully' }.

## POST /api/expenses/bulk-delete

**Auth:** Admin/Operator

**Purpose:** Bulk delete expenses.

**Body:** { ids: string[] }

**Response:** { message, deletedCount, requestedCount }.

# Section 10: Bills / Invoicing

## GET /api/bills

**Auth:** requireSession

**Purpose:** List bills (createdAt-desc). Supports billType, status, search filters.

**Body:** ?billType=, ?status= (draft|partial|paid|sent), ?search= (regex on billNumber/toName/toPhone)

**Response:** { bills: Bill[] }.

## POST /api/bills

**Auth:** requireSession

**Purpose:** Create bill. Generates billNumber = BILL-YYYYMM-####. Computes subTotal, taxableAmount, GST amounts, grandTotal, roundOff, balanceAmount. Auto-status. Pulls "from" defaults from Company. If customerId + paidAmount>0, auto-creates linked Payment.

**Body:** { billType?, date?, dueDate?, customerId?, fromName?, fromAddress?, fromGst?, fromPhone?, toName, toAddress?, toGst?, toPhone?, items: [{ description, hsn?, quantity, unit?, rate, amount? }], discountPercent?, discountAmount?, cgstPercent?, cgstAmount?, sgstPercent?, sgstAmount?, igstPercent?, igstAmount?, paidAmount?, paymentMode?, notes?, terms?, status? }

**Response:** { bill: Bill } (201).

### GET /api/bills/[id]

**Auth:** requireSession

**Purpose:** Fetch one bill.

**Response:** { bill: Bill } or 404.

### PUT /api/bills/[id]

**Auth:** requireSession

**Purpose:** Update bill. If items[] provided, recomputes all amounts. Keeps linked Payment mirror in sync.

**Body:** partial Bill fields

**Response:** { bill: Bill }.

### DELETE /api/bills/[id]

**Auth:** requireAdmin

**Purpose:** Delete bill. Cascades: deletes auto-synced Payment mirror. Manual Payments never touched.

**Response:** { message: 'Bill deleted successfully' }.

## Section 11: Excel Import

### GET /api/import

**Auth:** Open

**Purpose:** Returns the import template schema. Lists all 12 importable modules and their required fields.

**Response:** { modules: [{ id, label, fields: string[] }] }.

### POST /api/import

**Auth:** requireRole(['admin', 'operator'])

**Purpose:** Bulk-import rows for the specified module. Max 5000 rows. Server-side date normalization. Duplicate detection by natural key (duplicates skipped). Uses insertMany. Re-syncs Stock for touched production dates.

**Body:** { module: string, data: Array }

**Response:** { imported, skipped, duplicatesSkipped, errors: string[], skippedReasons: string[] }.

## Section 12: Admin & Debug

### GET /api/admin/fix-indexes

**Auth:** requireAdmin

**Purpose:** One-shot DB fix. Drops stale indexes from stocks and productions, recreates date\_-1 index. Idempotent.

**Response:** { collections: { stocks: { indexesBefore, stale, dropped, indexesAfter }, productions: {...} }, timestamp }.

### GET /api/admin/sync-all-stock

**Auth:** requireAdmin

**Purpose:** One-shot backfill. Finds all distinct production dates and re-syncs each into Stock collection.

**Response:** { timestamp, datesFound, dates, succeeded, failed, failures: [{ date, error }] }.

## GET /api/database

**Auth:** requireAdmin

**Purpose:** Export full backup of all 19 collections. User passwords STRIPPED.

**Response:** { version: 2, exportedAt, data: { ...collections... }, counts }.

## PUT /api/database

**Auth:** requireAdmin

**Purpose:** MERGE restore (not REPLACE). Same-\_id docs replaced, new-\_id inserted, missing docs preserved. Users collection SKIPPED. Uses bulkWrite replaceOne + upsert.

**Body:** Backup file (v2/unwrapped/raw shapes)

**Response:** { message, mode: "merge", counts: { inserted, replaced }, perCollection, errors? }.

## DELETE /api/database

**Auth:** requireAdmin

**Purpose:** Clears ALL transactional data (17 collections). Preserves only companies and users.

**Response:** { message, cleared: { customers: N, productions: N, ... } }.

## GET /api/database/clear-section

**Auth:** Admin

**Purpose:** Lists the 17 clearable collections with current counts.

**Response:** { sections: [{ key, label, count }] }.

## POST /api/database/clear-section

**Auth:** Admin

**Purpose:** Clears one specific collection.

**Body:** { collection: string }

**Response:** { message, collection, label, deletedCount }.

## GET /api/debug/sync

**Auth:** requireAdmin

**Purpose:** Runs syncStockForDate(date) and returns step-by-step debug info.

**Body:** ?date=YYYY-MM-DD REQUIRED

**Response:** { input\_date, steps: [{ step, ok, ... }] }.

## GET /api/debug/payment-sync

**Auth:** requireAdmin

**Purpose:** Diagnostic snapshot of Payment-CustomerPayment link state. Up to 20 recent records each, summary counts, unlinked payments.

**Response:** { summary, unlinkedPayments, recentPayments, recentCustomerPayments }.

## POST /api/debug/payment-sync/backfill

**Auth:** requireAdmin

**Purpose:** One-time backfill. For every Payment missing customerPaymentId, creates missing CustomerPayment mirror and links it.

**Response:** { totalScanned, successCount, failureCount, results: [{ paymentId, ok, error? }] }.

# Section 13: AI

## GET /api/ai/config

**Auth:** requireSession  
**Purpose:** Returns AI config with API key MASKED (last 4 chars only). Defaults if no config exists.  
**Response:** { provider: 'openai'|'groq', enabled, model, hasKey, keyMasked }.

**PUT /api/ai/config**

**Auth:** requireAdmin  
**Purpose:** Upserts AiConfig doc. Never echoes full key back (always masked).  
**Body:** { provider?, openaiApiKey? (empty clears), enabled?, model? }  
**Response:** Same shape as GET (masked).

**POST /api/ai/parse**

**Auth:** getSession (AI enabled)  
**Purpose:** Calls OpenAI/Groq with module system prompt + user text. temperature=0, max\_tokens=500, response\_format: json\_object, 12s timeout, 1 retry.  
**Body:** { module: string, text: string }  
**Response:** { fields: Record, raw: Record }. 403 if AI disabled. 502 if invalid JSON.

**POST /api/ai/test**

**Auth:** Admin  
**Purpose:** Test Connection button. Sends tiny ping to configured AI provider. 10s timeout, no retries.  
**Response:** { ok: true, provider, model, latencyMs, responsePreview, parsedOk, message } or { ok: false, error, status }.

Summary Tables

Authentication Quick Reference

| Who | Open    | (none)                                         |
|-----|---------|------------------------------------------------|
|     | Session | requireSession / getSession                    |
| Who | Admin   | requireAdmin or manual check                   |
|     | Role    | requireRole(['admin','operator'])              |
| Who | Role    | requireRole(['admin','operator','accountant']) |
|     | Role    | requireRole(['admin','accountant'])            |

Cross-Module Auto-Sync Web

| Effect | POST/PUT/DELETE /api/production/[id] | Re-syncs Stock s                   |
|--------|--------------------------------------|------------------------------------|
|        | POST /api/daily-sell (create)        | Mirrors into Cus<br>TractorPayment |
| Effect | PUT /api/daily-sell/[id]             | Cleans up old m                    |
|        | DELETE /api/daily-sell/[id]          | Cleans up linked                   |

|        |                                      |                  |
|--------|--------------------------------------|------------------|
| Effect | POST/PUT/DELETE /api/production/[id] | Re-syncs Stock s |
|        | POST /api/dispatch (create)          | Decrements Stoc  |
| Effect | DELETE /api/dispatch/[id]            | Restores Stock.c |
|        | POST /api/bills (create)             | If paidAmount>   |
| Effect | PUT /api/bills/[id]                  | Keeps Payment r  |
|        | DELETE /api/bills/[id]               | Cascades - delet |
| Effect | POST/PUT/DELETE /api/payments/[id]   | Mirrors into Cus |

## Special Query Parameters

|  |             |                      |                                               |
|--|-------------|----------------------|-----------------------------------------------|
|  | Description | ?search=             | /api/customers, /api/bills                    |
|  |             | ?page=, ?limit=      | /api/customers                                |
|  | Description | ?date=YYYY-MM-DD     | /api/production, /api/debug/sync              |
|  |             | ?all=true            | DELETE /api/production, /api/stock, /api/dail |
|  | Description | ?month=YYYY-MM       | /api/reports                                  |
|  |             | ?from=, ?to=         | /api/reports                                  |
|  | Description | ?type=               | /api/reports                                  |
|  |             | ?category=           | /api/expenses                                 |
|  | Description | ?date=from,to        | /api/expenses                                 |
|  |             | ?billType=, ?status= | /api/bills                                    |
|  | Description | ?key=                | /api/auth/init, /api/auth/reset-admin         |

## Bulk-Delete Pattern

All bulk-delete routes accept **{ ids: string[] }** and return **{ message, deletedCount, requestedCount }**. They use getSession() + manual role check so admin/operator can call while accountant gets 403. Modules with dedicated bulk-delete sub-routes: customers, production, orders, dispatch, customer-payment, labour-payment, tractor-payment, payments, dust-purchase, cement-purchase, hardner, electricity, factory-stuff, expenses, users. Exceptions: daily-sell uses POST /api/daily-sell with **{ ids }** body (admin-only branch); stock uses POST /api/stock with **{ ids }** body; bills have no bulk-delete route (admin must delete one at a time via DELETE /api/bills/[id]).

## Delete-All Pattern (Admin Only)

Several admin-only endpoints allow wiping entire collections. DELETE /api/production?all=true wipes all production entries and also wipes all stock entries since Stock is derived from Production. DELETE /api/stock?all=true wipes every stock entry independently. DELETE /api/daily-sell?all=true cleans up all linked mirrors before deleting. DELETE /api/database wipes all 17 transactional collections while preserving users and companies. POST /api/database/clear-section with **{ collection }** allows wiping one specific collection at a time.

## Pluralization Quirks

Response JSON keys have inconsistent pluralization that developers should be aware of: `/api/electricity` returns `{ electricity: [...] }` (not `electricities`), while other modules follow standard patterns like `{ customerPayments: [...] }`, `{ labourPayments: [...] }`, `{ hardners: [...] }`, and `{ factoryStuffs: [...] }`. Single-item responses always use the singular form: `{ customer: {...} }`, `{ hardner: {...} }`, `{ electricity: {...} }`, `{ factoryStuff: {...} }`, `{ dustPurchase: {...} }`, `{ cementPurchase: {...} }`, etc.